

Quadratic Regression using ANN.py

```
1import numpy as np
2import matplotlib.pyplot as plt
3import Activation as activation      # type: ignore
4import Dense as layer               # type: ignore
5import Loss as loss                 # type: ignore
6import sgd_optimizer as SGD         # type: ignore
7
8# -----
9# 1) Tạo dữ liệu bậc hai:  $y = a x^2 + b x + c + \text{noise}$ 
10# -----
11N = 1200
12rng = np.random.default_rng(42)
13
14true_a, true_b, true_c = 2.0, -3.0, 1.5
15X = rng.uniform(-3.0, 3.0, size=(N, 1))
16noise = rng.normal(0.0, 0.5, size=(N, 1))
17y = true_a * (X**2) + true_b * X + true_c + noise
18
19# chuẩn hoá đầu vào để training mượt hơn
20x_mean, x_std = X.mean(), X.std() + 1e-8
21Xn = (X - x_mean) / x_std
22
23# Sắp xếp để plot gọn hơn
24order = np.argsort(X[:, 0])
25X_plot = X[order]
26y_plot = y[order]
27
28# -----
29# 2) Xây mạng: 1-64-64-1 (ReLU, Linear)
30# -----
31Dense1 = layer.Dense(1, 64)
32act1 = activation.Activation_ReLU()
33
34Dense2 = layer.Dense(64, 64)
35act2 = activation.Activation_ReLU()
36
37Dense3 = layer.Dense(64, 1)
38act3 = activation.Activation_Linear()
39
40criterion = loss.Loss_MeanSquaredError()
41
42# SGD có decay + momentum
43optimizer = SGD.Optimizer_SGD_Decay_Momentum(
44    learning_rate=1.0,
45    decay=1e-5,
46    momentum=0.9
47)
48
```



```

49 # -----
50 # 3) Train loop (forward → loss → backward → update)
51 # -----
52 EPOCHS = 6000
53 print_every = 100
54 plot_every = 1000
55
56 for epoch in range(EPOCHS + 1):
57     # Forward
58     Dense1.forward(Xn)
59     act1.forward(Dense1.output)
60
61     Dense2.forward(act1.output)
62     act2.forward(Dense2.output)
63
64     Dense3.forward(act2.output)
65     act3.forward(Dense3.output)
66
67     # Loss + metric
68     loss_value = criterion.calculate(act3.output, y)
69     mae = np.mean(np.abs(act3.output - y))
70
71     if epoch % print_every == 0:
72         print(f"epoch {epoch:4d} | loss {loss_value:.4f} | MAE {mae:.4f} | LR
{optimizer.current_learning_rate:.6f}")
73     if epoch % plot_every == 0:
74         predictions = act3.output
75         y_pred_plot = predictions[order]
76         plt.scatter(X_plot, y_plot, s=5, label="data", color='pink')
77         plt.scatter(X_plot, y_pred_plot, s=1, label="ANN output", color='red')
78         plt.plot(X_plot, y_pred_plot, color='yellow')
79         plt.title(f"Epoch {epoch}")
80         plt.legend()
81         plt.show()
82
83     # Backward
84     criterion.backward(act3.output, y)
85     act3.backward(criterion.dinputs)
86
87     Dense3.backward(act3.dinputs)
88     act2.backward(Dense3.dinputs)
89
90     Dense2.backward(act2.dinputs)
91     act1.backward(Dense2.dinputs)
92
93     Dense1.backward(act1.dinputs)
94
95     # Update params (SGD + decay + momentum)
96     optimizer.pre_update_params()
97     optimizer.update_params(Dense1)

```



```

98     optimizer.update_params(Dense2)
99     optimizer.update_params(Dense3)
100     optimizer.post_update_params()
101
102 # -----
103 # 4) Ước lượng lại hệ số (a, b, c) từ đầu ra mô hình
104 #     Fit  $y_{\text{hat}} \approx A[a, b, c]^T$  với  $A = [x^2, x, 1]$ 
105 # -----
106 y_hat = act3.output # dự đoán cuối cùng của mạng
107
108 A = np.hstack([X**2, X, np.ones_like(X)])
109 theta, *_ = np.linalg.lstsq(A, y_hat, rcond=None)
110 a_hat, b_hat, c_hat = theta.flatten()
111
112 print(f"\nTrue:   a={true_a:.3f}, b={true_b:.3f}, c={true_c:.3f}")
113 print(f"Est.:   a={a_hat:.3f}, b={b_hat:.3f}, c={c_hat:.3f}")
114
115 # -----
116 # 5) Vẽ kết quả cuối cùng
117 # -----
118 Xs = np.linspace(X.min(), X.max(), 400).reshape(-1, 1)
119 Xs_n = (Xs - x_mean) / x_std
120
121 # forward để lấy y trên lưới
122 Dense1.forward(Xs_n); act1.forward(Dense1.output)
123 Dense2.forward(act1.output); act2.forward(Dense2.output)
124 Dense3.forward(act2.output); act3.forward(Dense3.output)
125 Ys_pred = act3.output
126
127 plt.figure()
128 plt.scatter(X, y, s=8, label="data", color='pink')
129 plt.plot(Xs, Ys_pred, linewidth=3, label="ANN prediction", color='yellow')
130
131 # đường cong bậc 2 khớp LS trên (a, b, c) từ y_hat
132 Ys_quad = a_hat * (Xs**2) + b_hat * Xs + c_hat
133 plt.plot(Xs, Ys_quad, linewidth=2, linestyle="--", label="ANN outputs", color='red')
134
135 plt.legend()
136 plt.title("Quadratic Regression using ANN")
137 plt.xlabel("x")
138 plt.ylabel("y")
139 plt.show()
140

```

